

Nén Trạng Thái

Zhou Wei

Trường Trung học 101 Trịnh Châu / Đại học Thiên Tân

Nguồn gốc: Nén trạng thái

Dynamic Programming / State Compression - Zhou Wei

Abstract

Nhiều bài toán có giá trị ứng dụng thực tế nhưng rất khó có thuật toán đa thức cho mọi dữ liệu. Tìm kiếm vét cạn luôn đúng nhưng thường quá chậm. Bài viết giới thiệu tư tưởng nén trạng thái qua một loạt ví dụ: dùng bit để biểu diễn tập hợp hoặc cấu hình cục bộ, rồi xây dựng truy hồi hoặc quy hoạch động trên những trạng thái đã nén.

Từ khóa. Nén trạng thái; băm; quy hoạch động; truy hồi; thao tác bit.

Contents

1	Mở đầu	1
1.1	Thao tác bit cơ bản	1
2	Khởi động: đặt xe trên bàn cờ	2
3	Mô hình bàn cờ	2
3.1	Xe với ô cấm	2
3.2	Đặt quân không kề nhau	2
3.3	Little Kings	3
3.4	Pháo binh	3
4	Mô hình phủ	3
4.1	Domino 1 x 2	3
4.2	Tăng tốc bằng lũy thừa ma trận	3
4.3	Các quân phủ khác	4
4.4	Bugs Integrated	4
5	Bản chất của nén trạng thái	4
6	Ví dụ đồ thị	4
6.1	Đếm ghép đầy đủ trong đồ thị hai phía	4
6.2	Đếm số thứ tự topo	4
6.3	Shortest Common Superstring	5

1 Mở đầu

Trong OI, ta gặp nhiều thuật toán đa thức như tìm kiếm nhị phân, Euclid, cây cân bằng, đường đi ngắn nhất. Tuy vậy cũng có những lớp bài như NPC hoặc NP-hard mà chưa biết thuật toán đa thức tổng quát: Hamilton cycle, TSP, đường đi đơn dài nhất trong đồ thị có hướng, v.v. Với những bài như vậy, nếu chỉ tìm kiếm thuần túy thì độ phức tạp có thể là $n!$ hoặc tệ hơn. Nén trạng thái là một cách thường dùng để có thuật toán mũ nhưng nhỏ hơn và dễ cài đặt hơn nhiều so với vét cạn trực tiếp.

1.1 Thao tác bit cơ bản

Các phép bit thường gặp:

Tên	C/C++	Pascal	Ý nghĩa
AND	&	and	tất cả là 1 thì 1
OR		or	có 1 thì 1
NOT	~	not	$0 \leftrightarrow 1$
XOR	^	xor	khác nhau thì 1

Một số ứng dụng nhỏ:

- Lấy một số bit của x : dùng AND với mặt nạ.
- Lấy bit 1 thấp nhất:

$$\text{lowbit}(x) = x \& (-x).$$

- Đặt một số bit thành 1: dùng OR với mặt nạ.
- Đảo một số bit: dùng XOR với mặt nạ.

2 Khởi động: đặt xe trên bàn cờ

Bài dẫn. Trên bàn cờ $n \times n$, $n \leq 20$, đặt n quân xe sao cho không quân nào ăn quân nào. Đếm số cách đặt.

Đây là một bài tổ hợp đơn giản, đáp án là $n!$. Nhưng để giới thiệu nén trạng thái, ta giải theo hướng truy hồi. Đặt từng hàng một; một trạng thái là tập các cột đã có quân xe. Nếu cột đã dùng thì bit tương ứng là 1, ngược lại là 0. Ví dụ với $n = 5$, trạng thái 01101₂ nghĩa là các cột 1, 3, 4 đã dùng.

Gọi f_s là số cách đặt tới trạng thái s . Khi $s = 01101_2$, trạng thái này có thể đến từ ba trạng thái trước đó bằng cách bỏ lần lượt bit ở cột 1, 3, 4:

$$f_{01101} = f_{01100} + f_{01001} + f_{00101}.$$

Tổng quát:

$$f_0 = 1, \quad f_s = \sum_{i: \text{bit } i \text{ của } s = 1} f_{s-2^i}.$$

Thuật toán có độ phức tạp $\mathcal{O}(n2^n)$ và bộ nhớ $\mathcal{O}(2^n)$. Nó kém hơn công thức $n!$, nhưng có khả năng mở rộng tốt hơn.

3 Mô hình bàn cờ

3.1 Xe với ô cấm

Nếu một số ô không được đặt quân xe, công thức tổ hợp không còn rõ ràng, nhưng truy hồi nén trạng thái chỉ cần sửa rất ít. Với hàng r , dùng mặt nạ a_r để đánh dấu các cột có thể đặt. Khi trạng thái hiện tại là s , ta chỉ liệt kê các bit 1 trong

$$s \& a_r,$$

nhờ vậy không cần kiểm tra từng cột trong vòng lặp. Đây là ví dụ điển hình cho việc dùng bit mask để biến ràng buộc cục bộ thành phép toán hằng số.

3.2 Đặt quân không kề nhau

Cho bàn cờ $n \times m$, $n, m \leq 80$, $nm \leq 80$. Đặt $k \leq 20$ quân sao cho không có hai quân kề nhau theo hàng hoặc cột. Vì $nm \leq 80$, ít nhất một trong hai chiều không vượt quá 8; chọn chiều đó làm số cột.

Ta tiền xử lý mọi trạng thái hợp lệ của một hàng vào mảng $s_1, \dots, s_r$, tức là các mask không có hai bit 1 liền nhau. Gọi c_j là số bit 1 của trạng thái s_j . Đặt

$$f_{i,j,k}$$

là số cách xét xong i hàng, hàng i dùng trạng thái s_j , và đã đặt tổng cộng k quân. Khi hàng hiện tại s_j không xung đột với hàng trước s_p , tức

$$s_j \& s_p = 0,$$

ta có

$$f_{i,j,k} = \sum_p f_{i-1,p,k-c_j}.$$

Có thể dùng mảng cuộn vì chỉ phụ thuộc hàng trước.

3.3 Little Kings

Với bài đặt k quân vua trên bàn $n \times n$, $n \leq 10$, quân vua ăn theo cả tám hướng. Trạng thái một hàng vẫn là mask không có hai bit 1 kề nhau. Khác biệt là hàng hiện tại còn không được chéo với hàng trước. Có thể tiền xử lý một mask q_j biểu diễn những ô ở hàng trước bị cấm khi hàng hiện tại là s_j . Điều kiện tương thích là

$$q_j \& s_p = 0.$$

Công thức DP giữ nguyên dạng như trên.

3.4 Pháo binh

Trong bài *Artillery Positions*, bàn cờ $n \times m$, $n \leq 100$, $m \leq 10$, có một số ô cấm. Cần đặt nhiều quân nhất sao cho trên cùng hàng hoặc cột, hai quân cách nhau ít nhất hai ô trống.

Tầm ảnh hưởng theo hàng/cột là 2, nên trạng thái cần nhớ hai hàng. Gọi

$$f_{i,j,k}$$

là số quân tối đa sau i hàng, trong đó hàng i có trạng thái s_j và hàng $i-1$ có trạng thái s_k . Khi ba trạng thái s_j, s_k, s_l đôi một không xung đột và s_j không đặt vào ô cấm, ta chuyển:

$$f_{i,j,k} = \max_l f_{i-1,k,l} + c_j.$$

Vì số trạng thái hợp lệ của một hàng nhỏ, thuật toán $\mathcal{O}(nt^3)$ đủ nhanh. Có thể tiền xử lý các cặp hoặc bộ trạng thái tương thích để giảm hằng số.

4 Mô hình phủ

4.1 Domino 1 x 2

Cho bàn $n \times m$, $1 \leq n, m \leq 11$, dùng domino 1×2 phủ kín bàn. Giả sử $m \leq n$. Gọi $f_{i,s}$ là số cách đã phủ kín $i - 1$ hàng, và hàng i đang có trạng thái phủ s . Ta DFS mọi cách đặt domino ở hàng hiện tại để sinh cặp trạng thái (s_1, s_2) : trạng thái để lại cho hàng hiện tại và trạng thái yêu cầu từ hàng trước. Công thức:

$$f_{0,2^m-1} = 1, \quad f_{i,s_1} = \sum f_{i-1,s_2},$$

trong đó (s_1, s_2) là một cách đặt hợp lệ.

Cách DFS tự nhiên có ba lựa chọn tại một vị trí: đặt domino dọc, đặt domino ngang, hoặc không đặt ô đó tùy theo trạng thái đã phủ. Số cách sinh ra không phải 4^m ; nó thỏa truy hồi gần với

$$h_0 = 1, \quad h_1 = 2, \quad h_i = 2h_{i-1} + h_{i-2},$$

nên thực tế nhỏ hơn rất nhiều so với liệt kê thô.

4.2 Tăng tốc bằng lũy thừa ma trận

Nếu n rất lớn, DP theo từng hàng sẽ chậm. Ta xây đồ thị có 2^m đỉnh, mỗi đỉnh là một trạng thái hàng. Nếu (s_1, s_2) là một chuyển hợp lệ, thêm cạnh từ s_2 tới s_1 . Khi đó số cách đi từ trạng thái đầy đủ $(2^m - 1)$ về chính nó sau n bước là đáp án. Với ma trận kề G , đáp án là phần tử tương ứng của G^n .

Ma trận này rất thưa, nên có thể lưu các phần tử khác 0 bằng bộ ba $(p, q, value)$ và nhân ma trận thưa: với mỗi phần tử khác 0 của A , tìm các phần tử cùng hàng phù hợp trong B , rồi cộng vào kết quả. Nhờ đó trong nhiều trường hợp thực tế tốt hơn rất nhiều so với nhân ma trận đặc $\mathcal{O}(8^m)$.

4.3 Các quân phủ khác

Với domino 1×2 và quân chữ L 2×2 bỏ một ô, trạng thái và công thức giống domino, nhưng DFS sinh chuyển có nhiều trường hợp hơn. Với quân $1 \times r$, trạng thái có thể phải ghi nhớ $r - 1$ hàng. Nếu lưu trực tiếp nhiều mask nhị phân sẽ có nhiều trạng thái dư thừa; tốt hơn là dùng hệ cơ số r . Mỗi chữ số mô tả số ô liên tiếp đã được phủ theo chiều dọc tại cột tương ứng. Đây là ví dụ cho thấy “nén trạng thái” không nhất thiết chỉ là nhị phân.

4.4 Bugs Integrated

Với bàn $n \times m$, $n \leq 150$, $m \leq 10$, cần đặt nhiều quân 2×3 nhất, một số ô bị cấm. Dùng hệ tam phân để biểu diễn trạng thái của hai hàng: chữ số 0, 1, 2 mô tả mức đã phủ liên tục từ hàng trên xuống tại cột đó. Đặt $f_{i,s}$ là số quân tối đa tới hàng i với trạng thái s . DFS tại từng hàng sinh chuyển từ trạng thái hàng trước sang hàng hiện tại, thử đặt quân ngang hoặc dọc nếu không chạm ô cấm, đồng thời có thể đặt các “ô ảo” để đóng những vị trí không dùng. Công thức tổng quát:

$$f_{i,s_1} = \max_s \{f_{i-1,s} + num(s \rightarrow s_1)\}.$$

5 Bản chất của nén trạng thái

Các ví dụ trên đều là những truy hồi hoặc DP thông thường, chỉ có một hoặc nhiều chiều trạng thái được “nén” thành số nguyên. Có thể xem đây là một dạng bấm: một tập hợp, một cấu hình hàng, hoặc một trạng thái phủ được mã hóa bằng bit hoặc chữ số trong một cơ sở nhỏ.

Động cơ của nén trạng thái thường là: trạng thái quá thô không đủ để chuyển đúng. Ví dụ, chỉ biết đã xử lý tới hàng i là không đủ để phủ domino, vì hàng kế tiếp còn bị ảnh hưởng bởi các ô nhỏ xuống từ hàng trước. Ta cần ghi thêm cấu hình biên; nén trạng thái giúp cấu hình đó đủ nhỏ để liệt kê.

6 Ví dụ đồ thị

6.1 Đếm ghép đầy đủ trong đồ thị hai phía

Có $n \leq 20$ con bò và $m \leq 20$ đồng cỏ. Mỗi con bò thích một số đồng cỏ và không muốn chung đồng cỏ với bò khác. Hỏi có bao nhiêu cách gán mỗi con bò một đồng cỏ nó thích.

Dùng một mask s biểu diễn tập đồng cỏ đã dùng. Gọi $z(s) = \text{popcount}(s)$ là số bò đã xử lý. Nếu ma trận kề là g , trong đó $g_{i,j} = 1$ khi bò i thích đồng cỏ j , thì

$$f_0 = 1, \quad f_s = \sum_{j: \text{bit } j \text{ của } s = 1} f_{s-2^j} g_{z(s),j}.$$

Đáp án là tổng f_s trên các trạng thái có $\text{popcount}(s) = n$. Có thể hiểu tương đương theo tập hợp: f_s là số cách gán các bò đầu tiên vào đúng các đồng cỏ thuộc tập S ; để thu nhỏ bài toán, liệt kê đồng cỏ của con bò cuối.

6.2 Đếm số thứ tự topo

Cho đồ thị có hướng $n \leq 20$, cần đếm số thứ tự topo hợp lệ. Trạng thái S là tập đỉnh đã đặt vào thứ tự. f_S là số thứ tự topo của các đỉnh trong S . Ta có thể liệt kê đỉnh cuối cùng của S ; đỉnh đó hợp lệ nếu mọi cạnh đi ra từ nó tới trong S đều đã được xử lý phù hợp, hoặc tương đương khi bỏ nó khỏi S , các điều kiện tiên nhiệm vẫn đúng. Chi tiết phụ thuộc cách lưu cạnh, nhưng mô hình vẫn là DP tập con kinh điển.

6.3 Shortest Common Superstring

Cho $n \leq 15$ xâu, cần tìm xâu ngắn nhất chứa mọi xâu đã cho làm xâu con; nếu có nhiều đáp án, chọn từ điển nhỏ nhất. Trước hết loại xâu nào là xâu con của xâu khác. Sau đó tiên xử lý độ chồng lớn nhất giữa mọi cặp xâu. Trạng thái

$$f_{S,i}$$

là xâu ngắn nhất tạo từ tập S và kết thúc bằng xâu i . Chuyển từ $f_{S-\{i\},j}$ sang $f_{S,i}$ bằng cách nối thêm phần chưa chồng của i . Đây là một ví dụ khác: trạng thái là tập xâu đã dùng, nén bằng mask.

7 Kết luận

Nén trạng thái không phải là một thuật toán riêng lẻ, mà là cách biểu diễn trạng thái. Khi số phần tử nhỏ, hoặc khi chỉ cần nhớ một “biên” hẹp của bài toán, bit mask và cơ sở nhỏ giúp biến tìm kiếm thô thành truy hồi hoặc quy hoạch động có cấu trúc. Điểm quan trọng là chọn đúng phần thông tin cần lưu: đủ để chuyển trạng thái không có hậu hiệu, nhưng không dư thừa đến mức làm nổ số trạng thái.